

Avance 2, Prueba Ecostress

Natalia Hernández-López^{ID*} y Sebastián Cruz-Solano

*Escuela de Ingeniería Electrónica, Instituto Tecnológico de Costa Rica (ITCR), 30101 Cartago, Costa Rica, {natih1600, scrusz100}@estudiantec.cr

Resumen—El presente desarrolla el proceso de investigación, análisis y diseño para el modelado de una aplicación de detección/corrección de errores, preprocesamiento de señales y esquemas de modulación. Los datos provienen de una prueba de ecostress, estos deben de ser preprocesados y filtrados para luego emular su transmisión por un canal con ruido Gaussiano.

Palabras Clave—Antena, AWGN, Hamming, nRF52832.

I. INTRODUCCIÓN

En este proyecto se busca la aplicación e integración de distintas áreas de conocimiento para el modelado de un sistema de radiofrecuencia con corrección y detección de errores para una aplicación médica mediante un SoC nRF52832 Nordic Semiconductor. La finalidad de este proyecto es simular el flujo de este procedimiento al recibir los datos y tener que procesarlos para su envío. El desarrollo del mismo se realizará en 2 etapas, en las que se trabajará en el desarrollo de los módulo y bloques necesarios para lograr este objetivo, además, se realizarán distintos análisis teóricos e investigación de ciertos componentes del proyecto. Para trabajar en el proyecto se debe utilizar herramientas como MATLAB, EAGLE, Phytion, Jupyter Notebook y Git. Para este segundo avance, se trabajó con el bloque de transmisión que incluía el sub-bloque codificación con codificación de Hamming (7,4) y (15,11). Además, se debió de realizar la modulación y la investigación de la antena. También, un factor muy importante es el diseño del modelo del canal con ruido AWGN para la transmisión. Finalmente, también se desarrolló el bloque del receptor con el código de demodulación, decodificación y visualización.

II. MÓDULO CODIFICADOR

La etapa de codificación tiene como objetivo codificar en una representación binaria de 8 bits la información de BPM para transmitir a través del canal ruidoso AWGN. Para esto se implementó la codificación de Hamming en dos variantes, Hamming (7,4) y Hamming (15,11).

II-A. Codificación Hamming (7,4)

El código Hamming (7,4) toma un bloque de cuatro bits de datos y genera una palabra codificada de siete bits mediante la inserción de tres bits de paridad. En este sistema, cada muestra de BPM es de 8 bits, por lo que se divide en dos nibbles de 4 bits, generando dos palabras codificadas independientes. Para cada nibble se calculan los bits de paridad p_1 , p_2 y p_3 utilizando las relaciones:

$$p_1 = d_0 \oplus d_1 \oplus d_3, \quad (1)$$

$$p_2 = d_0 \oplus d_2 \oplus d_3, \quad (2)$$

$$p_3 = d_1 \oplus d_2 \oplus d_3, \quad (3)$$

donde d_i representan los bits de datos. La palabra codificada resultante se organiza en el orden:

$$[p_1, p_2, d_0, p_3, d_1, d_2, d_3].$$

De esta forma, con la codificación de Hamming (7,4) cada dato de BPM genera dos words de siete bits, produciendo una expansión total de 14 bits por dato. Una vez generada la totalidad de codewords, esta secuencia se almacena en el archivo `codificacion_hamming_7_4.txt`, el cual es posteriormente utilizado como entrada del modulador GFSK en MATLAB.

II-B. Codificación Hamming (15,11)

La codificación de Hamming (15,11) genera bloques de once bits de datos con cuatro bits de paridad, generando palabras codificadas de quince bits. Debido a que cada muestra de BPM es de 8 bits, se debe añadir tres bits de relleno en cero para completar los once bits de información requeridos por el código.

Los bits de paridad p_1 , p_2 , p_4 y p_8 se calculan siguiendo las relaciones:

$$p_1 = d_0 \oplus d_1 \oplus d_2 \oplus d_3 \oplus d_4 \oplus d_5 \oplus d_6 \oplus d_7 \oplus d_8 \oplus d_9 \oplus d_{10}, \quad (4)$$

$$p_2 = d_0 \oplus d_1 \oplus d_3 \oplus d_4 \oplus d_6 \oplus d_7 \oplus d_9 \oplus d_{10}, \quad (5)$$

$$p_4 = d_0 \oplus d_2 \oplus d_3 \oplus d_5 \oplus d_7 \oplus d_8 \oplus d_9 \oplus d_{10}, \quad (6)$$

$$p_8 = d_0 \oplus d_1 \oplus d_2 \oplus d_3 \oplus d_4 \oplus d_5 \oplus d_6 \oplus d_7. \quad (7)$$

Obteniendo la palabra final de quince bits que se exporta al archivo `codificacion_hamming_15_11.txt`.

$$[p_1, p_2, d_0, p_4, d_1, d_2, d_3, p_8, d_4, d_5, d_6, d_7, d_8, d_9, d_{10}].$$

III. MÓDULO CANAL

Este bloque tiene como función principal modelar de una manera aproximada el canal de comunicación, ya que este no cuenta con un modelo preestablecido de canal ni parámetros S. Para modelar las características se utiliza un ruido de tipo AWGN. Para aplicar este ruido se utilizó el método de Box-Muller el cual da como resultados un conjunto de datos con desviación estándar de 1 y un valor promedio de 0. Para

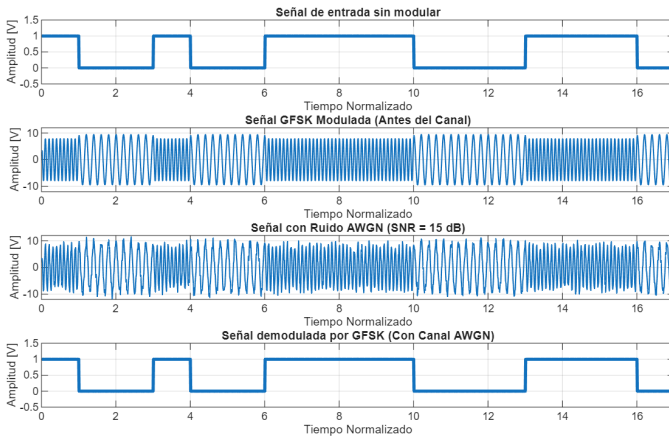


Figura 1. Señales de entrada y salidas del canal

este módulo la librería más importante fue `numpy` ya que en este se encuentran las funciones `isrealobj`, `sqrt` y `standard_normal`.

El módulo del canal se compone de 3 funciones: `addNoise`, `transmitSignal` y `transmitReport`. La función principal es `transmitSignal`, esta tiene como entradas el vector de valores binarios provenientes de Hamming, la atenuación del canal y la relación SNR en decibelios; en esta se llama la función `addNoise` la cual se encarga de aplicar el método de Box-Muller al generar un vector de valores de ruido, el cual procede a agregarse a la señal atenuada. Por último la función `transmitReport` se encarga de crear reportajes de los datos enviados y figuras como se observa en 1.

IV. BLOQUE DEL CANAL EN PYTHON

El bloque de canal implementado en Python tiene como objetivo modelar los efectos que sufre la señal modulada durante la transmisión inalámbrica. El modelo desarrollado incluye dos efectos principales, la adición de ruido blanco gaussiano aditivo (AWGN) y la presencia de trayectorias múltiples (*multipath*). A partir de este modelo se obtiene una versión degradada de la señal modulada en GFSK, la cual es posteriormente demodulada y decodificada.

El bloque de canal en Python utiliza las muestras complejas producidas por el modulador GFSK implementado en MATLAB que fueron almacenadas en los archivos `modulado_hamming_7_4.txt` y `modulado_hamming_15_11.txt`. Para leer estos archivos se desarrolló una función con la biblioteca `pandas` que da como resultado un `DataFrame` en la que la primera columna contiene la secuencia de muestras complejas correspondientes a la señal modulada sin ruido. Sobre esta señal se aplican las afectaciones del canal de ruido blanco gaussiano aditivo (AWGN) y la presencia de multitrayectoria.

Para modelar el ruido del canal se implementó una función `AWGN_generator` que utiliza el método de Box-Muller

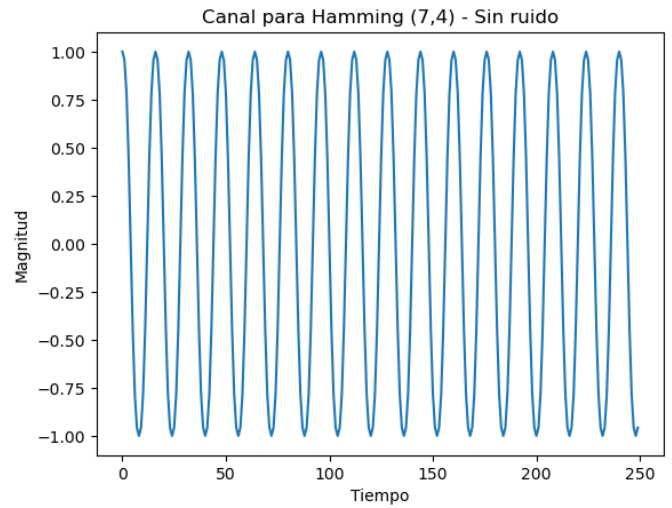


Figura 2. Canal para Hamming (7,4) - Sin ruido

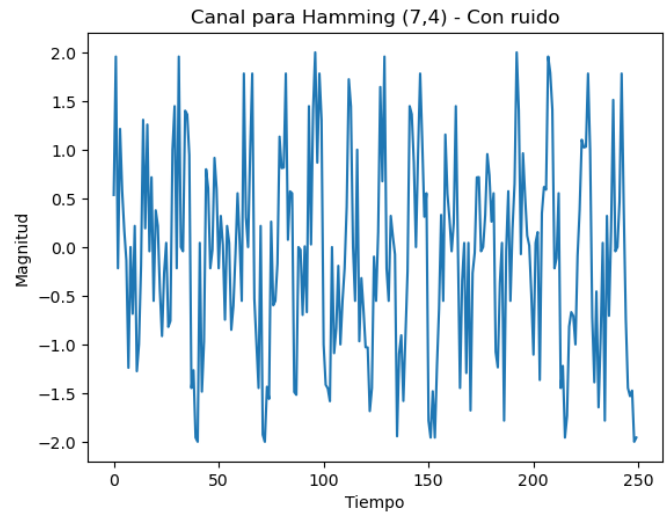


Figura 3. Canal para Hamming (7,4) - Con ruido

para generar muestras de ruido con distribución gaussiana de media cero y varianza unitaria. Después, la función `agregar_ruido` recorre todas las muestras complejas del `DataFrame` y, para cada elemento construye la muestra ruidosa. Obteniendo así una señal compleja que modela la salida de un canal con ruido AWGN.

Además del ruido aditivo, el canal introduce distorsiones debidas al fenómeno de multitrayectoria, donde la señal llega al receptor por varios caminos con diferentes retardos y atenuaciones. Para simular este comportamiento, se implementó la función `simular_multipath`, que genera, para cada fila de datos, una señal compuesta por la suma de varias copias retardadas y escaladas de la señal original:

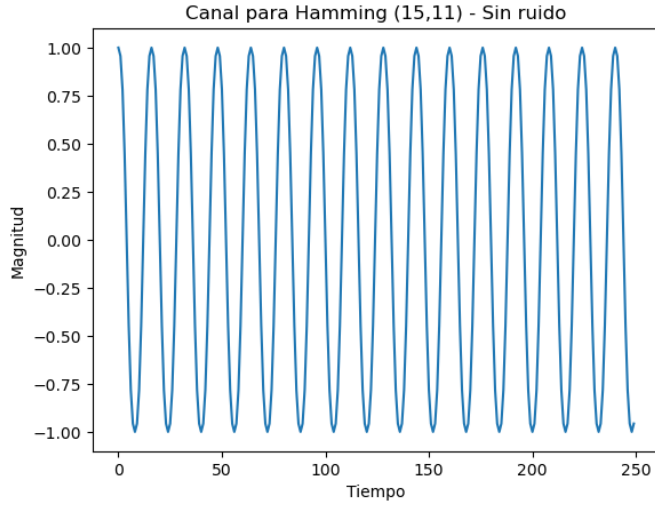


Figura 4. Canal para Hamming (15,11) - Sin ruido

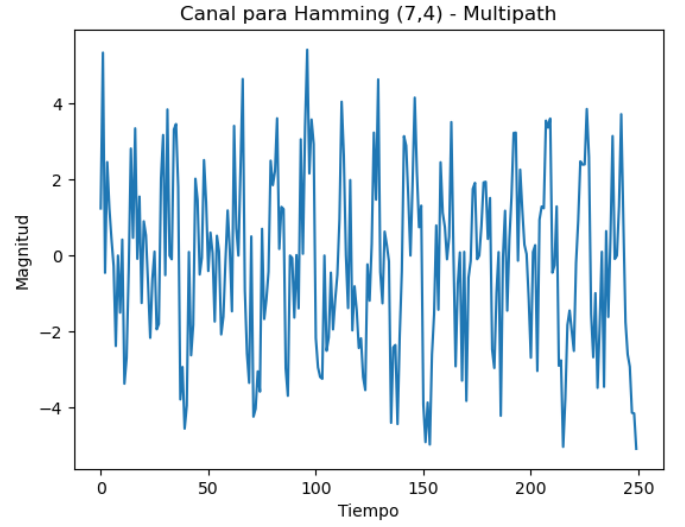


Figura 6. Canal para Hamming (7,4) - Multipath

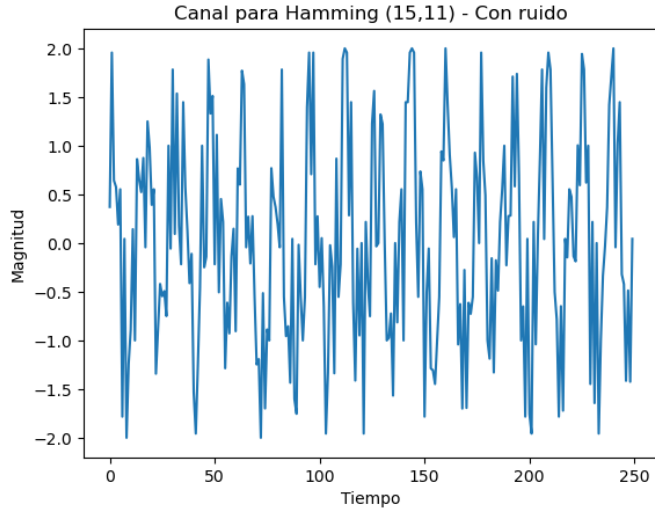


Figura 5. Canal para Hamming (7,4) - Con ruido

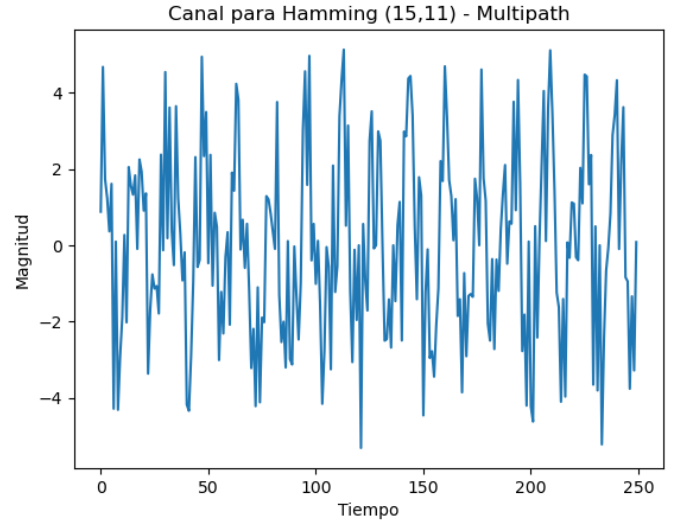


Figura 7. Canal para Hamming (15,11) - Multipath

$$y_k = \sum_{i=1}^{N_{\text{cam}}} \alpha_i x_{k-\tau_i}, \quad (8)$$

donde α_i es la ganancia aleatoria asociada al i -ésimo camino, y τ_i es el retraso en número de muestras. Los retardos se seleccionan de manera aleatoria dentro de un rango acotado, mientras que las ganancias se eligen en un intervalo típico entre 0,5 y 1,0, simulando diferentes niveles de atenuación en cada trayectoria.

Adicionalmente, se incluye el cálculo de la capacidad del canal mediante la ecuación de Shannon:

$$C = B \log_2(1 + \text{SNR}), \quad (9)$$

donde B es el ancho de banda del sistema y SNR es la relación señal a ruido lineal. Esta capacidad se compara con la tasa de transmisión del dispositivo nRF52832 de 1 Mbps para determinar si existe saturación del canal.

V. DEMODULADOR

Aquí se realizó el proceso inverso que en la modulación, se tiene la misma secuencia binaria modulada y se realiza el código en Matlab para obtener el dato en binario, igualmente se graficó la señal modulada y demodulada para observar el funcionamiento, los resultados se muestran en la señal más abajo Figura 1.

VI. BLOQUE DECODIFICADOR

El bloque de decodificación tiene como objetivo recuperar los bits de información originales a partir de las palabras recibidas desde el demodulador GFSK, corrigiendo los errores introducidos por el canal y reconstruyendo los valores finales de latidos por minuto (BPM). Este módulo implementa los decodificadores de Hamming(7,4) y Hamming(15,11) con las funciones `hamming_7_4_decode` y `hamming_15_11_decode`, respectivamente.

La entrada del decodificador consiste en un vector unidimensional de bits $\{0,1\}$ generado por el demodulador que se dividen en bloques de tamaño fijo, 7 bits para Hamming (7,4) y 15 bits para Hamming (15,11).

Para cada palabra recibida se calcula el síndrome, el cual permite detectar y localizar errores de un solo bit. El síndrome se obtiene aplicando las ecuaciones de paridad definidas en el codificador, para Hamming(7,4), el síndrome está dado por:

$$S = s_1 + 2s_2 + 4s_3,$$

mientras que para Hamming(15,11) es:

$$S = s_1 + 2s_2 + 4s_4 + 8s_8.$$

Un síndrome igual a cero indica que la palabra no contiene errores detectables; pero si es distinto de cero, el bit ubicado en la posición indicada por el síndrome S es invertido para corregir la palabra.

Una vez corregida la palabra, se extraen únicamente los bits de información, que en Hamming (7,4) son los que están en las posiciones 3, 5, 6 y 7 de la palabra recibida, mientras que en Hamming (15,11) son las posiciones 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15. Los bits recuperados se concatenan en un vector de datos que equivale a la información binaria original antes del proceso de codificación.

Posteriormente, como cada BPM fue representado en 8 bits, el vector de datos recuperado se convierte a un entero para reconstruir los valores de BPM decodificados, que son almacenados en los archivos de texto `BPM_decodificados_7_4.txt` y `BPM_decodificados_15_11.txt`.

VII. BLOQUE VISUALIZACIÓN

El bloque de visualización tiene como objetivo analizar y presentar los resultados de los datos de BPM después de los procesos de transmisión y decodificación de los datos.

Para esto, primero se cargan los datos originales de la prueba Ecostress desde el archivo `Dataset.xlsx`, extrayendo la columna de latidos por minuto (BPM) y la columna de tiempos asociados. De forma paralela, se leen los valores de BPM reconstruidos a partir de los bits decodificados para las codificaciones de Hamming (7,4) y Hamming (15,11).

Con estos datos se graficó una curva comparativa de *BPM original vs. BPM decodificado*, que se muestra en la Figura 8; en la cual se observa la recuperación de los datos tras el proceso de codificación, modulación, canal ruidoso y demodulación.

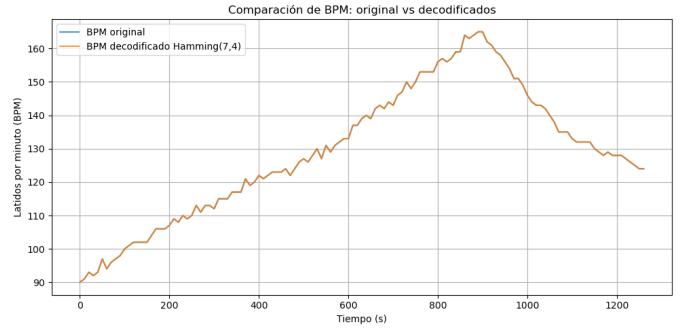


Figura 8. Comparación de BPM: original vs decodificados

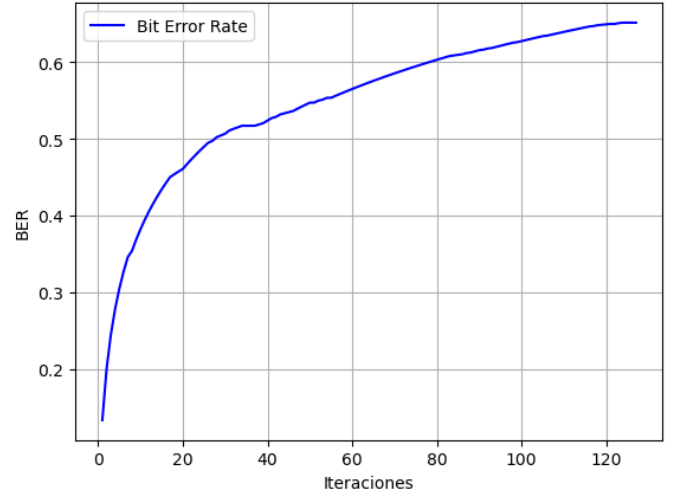


Figura 9. Bit Error Rate (BER)

Adicionalmente, se clasifican la frecuencias cardíacas en zonas de entrenamiento en función de la frecuencia cardíaca máxima obtenida en los datos. Estas zonas se definen mediante porcentajes de la frecuencia máxima y se resumen en la Tabla I.

Cuadro I
TABLA RESUMEN DE ZONAS DE FRECUENCIA CARDÍACA

Zona	Descripción	Rango de FC [BPM]
Zona 1	Calentamiento	< 107.25 bpm
Zona 2	Quema de grasa	107.25 – 132.00 bpm
Zona 3	Aeróbica	132.00 – 140.25 bpm
Zona 4	Anaeróbica	140.25 – 156.75 bpm
Zona 5	Esfuerzo máximo	> 156.75 bpm

Finalmente, en la Figura 9 se genera una gráfica del *Bit Error Rate* (BER) en función de la cantidad de iteraciones. Esta curva permite cuantificar el desempeño del sistema de comunicación y comparar el efecto del uso de distintos códigos de Hamming sobre la probabilidad de error en la transmisión.

REFERENCIAS

- [1] muRata, "Microwave coaxial connectors," Nov 2018, [Online] Available: <https://www.mouser.com/datasheet/2/281/o33e-844855.pdf>.
- [2] Bluetooth, "Bluetooth technology overview," 2022, [Online] Available: <https://www.bluetooth.com/learn-about-bluetooth/tech-overview/>.
- [3] NXP, "Ble antenna design guide," May 2018, [Online] Available: <https://www.nxp.com/docs/en/user-guide/UM10992.pdf>.
- [4] R. Coto-Salazar and R. Rimolo-Donadio, "Design of a 4g/lte multiband antenna considering curvature effects," in *2018 IEEE MTT-S Latin America Microwave Conference (LAMC 2018)*, 2018, pp. 1–3.
- [5] PCBgogo, "What is the difference between fr-4 material and rogers material?" Aug 2019, [Online] Available: <https://www.pcbgogo.com>.
- [6] Jan 2020. [Online]. Available: <https://www.geeksforgeeks.org/hamming-code-implementation-in-python/>
- [7] L. García-Cervantes, "¿Qué son las zonas de entrenamiento?," Ene 2020, [Online] Available: <https://www.triatlonnoticias.com/entrenamientos-triatlon/que-son-las-zonas-de-entrenamiento/>.